Trie Operations

Trie is the core component of the autocomplete system.

To maintain the trie efficiently, we must support the following operations:

- Create
- Update
- Delete

Each operation is important for maintaining accurate and fast autocomplete suggestions.

1. Create Operation

Trie creation is performed periodically by workers.

The data source comes from:

- Analytics Logs
- Aggregated Data
- Analytics Database

Trie Creation Flow

Step 1

Users generate search queries.

Step 2

Queries are stored in analytics logs.

Step 3

Aggregators process raw logs into summarized frequency data.

Step 4

Workers read aggregated data.

Step 5

Workers build a new trie structure.

Step 6

The new trie is stored in:

- Trie DB
- Trie Cache

Why Workers Build Trie

Trie construction is computationally expensive.

Therefore:

- It is performed asynchronously
- It runs periodically
- It does not affect query serving latency

This design improves:

- Scalability
- Reliability
- Performance

Example

Suppose aggregated data contains:

Query	Frequency
tree	1200
try	980
true	1500

Workers build trie nodes and cache top-k results for each prefix.

2. Update Operation

Autocomplete data changes over time because:

- New trends emerge
- Search popularity changes
- User behavior evolves

Therefore, trie data must be updated periodically.

There are two approaches.

Option 1: Rebuild Trie Periodically (Preferred)

In this approach:

1. Workers build an entirely new trie.
2. The new trie replaces the old trie.

Example:

```
None
Weekly Trie Rebuild
```

Why This Approach is Preferred

Advantages:

- Simple implementation
- Consistent trie structure
- No partial updates
- Easier synchronization
- Better performance at large scale

This is the most common production approach.

Workflow

Step 1

Build new trie in background.

Step 2

Validate trie.

Step 3

Swap old trie with new trie atomically.

Step 4

Refresh Trie Cache.

This minimizes downtime and inconsistency.

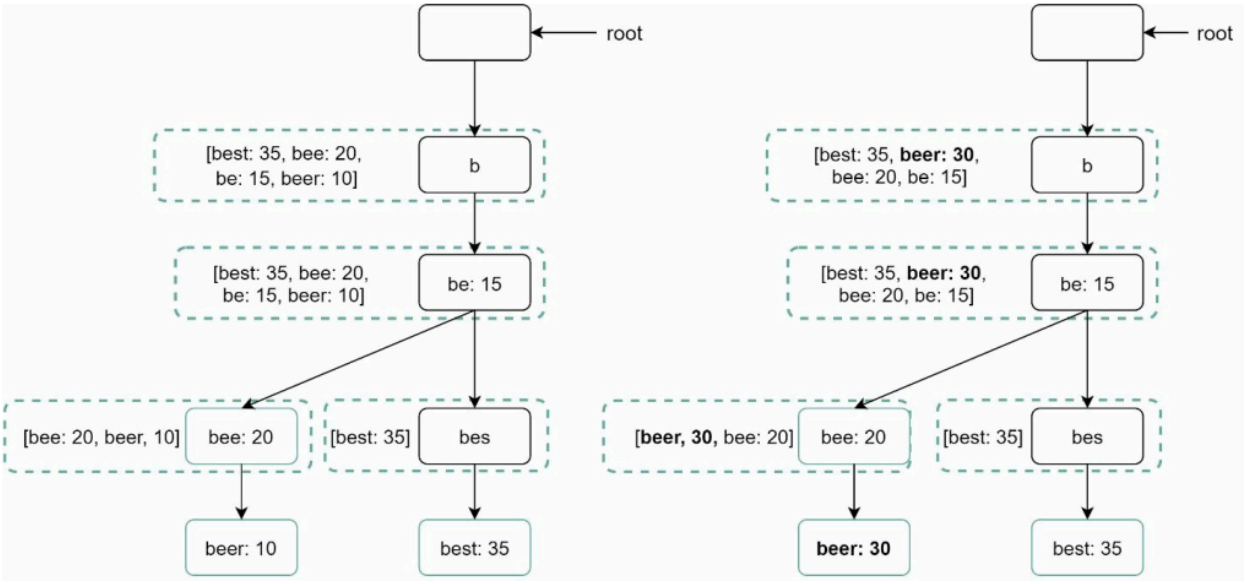
Option 2: Update Individual Trie Nodes

Instead of rebuilding the entire trie, update only modified nodes.

Example:

- Query `"beer"` frequency changes from:

```
None
10 → 30
```



This approach is shown in Figure.

Problem with Direct Updates

Each trie node stores:

- Top queries of its subtree

Therefore:

- Updating one node affects all ancestor nodes.

Example

Suppose:

```
None
beer = 10
```

Updated to:

```
None
beer = 30
```

Then the following nodes must also be updated:

- beer
- bee
- be
- b
- root

because ancestor nodes cache top-k search queries.

Why Direct Updates Are Expensive

Disadvantages:

- Multiple node updates required
- Complex synchronization
- High write amplification
- Poor scalability for large tries

Therefore:

- Direct updates are generally avoided in large-scale systems.

However:

- Acceptable for small trie sizes.

Comparison of Update Approaches

Approach	Advantages	Disadvantages
Periodic rebuild	Simple, scalable	Slightly stale data
Direct node update	Real-time updates	Expensive and complex

3. Delete Operation

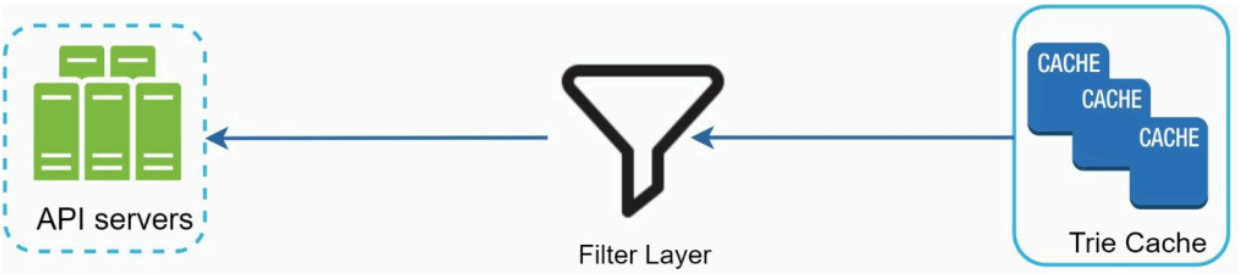
Autocomplete systems must remove harmful or inappropriate suggestions.

Examples:

- Hate speech
- Violent content
- Sexually explicit queries
- Dangerous suggestions
- Spam content

Filter Layer

To support deletion efficiently, a filter layer is added before Trie Cache.



This is shown in Figure.

Query Flow with Filter Layer

Step 1

User sends autocomplete request.

Step 2

API server fetches suggestions from Trie Cache.

Step 3

Filter layer checks results against filtering rules.

Step 4

Blocked suggestions are removed.

Step 5

Safe results are returned to the client.

Advantages of Filter Layer

Benefits:

- Flexible filtering rules
- Real-time filtering
- Easy moderation
- Fast response

This avoids rebuilding the trie immediately after every moderation request.

Asynchronous Physical Deletion

Initially:

- Suggestions are filtered logically.

Later:

- Workers remove unwanted suggestions physically from the database asynchronously.

This cleaned dataset is used during the next trie rebuild cycle.

Why Asynchronous Deletion is Better

Advantages:

- No impact on query latency
- Faster moderation response
- Avoids immediate trie reconstruction

Overall Trie Lifecycle

Create

- Workers build trie from aggregated analytics data.

Update

- Trie rebuilt periodically OR nodes updated directly.

Delete

- Filter layer blocks unwanted results.
- Physical deletion occurs asynchronously.

Complete Trie Architecture

Component	Responsibility
Analytics Logs	Store raw queries
Aggregators	Summarize frequency data
Workers	Build/update trie
Trie DB	Persistent storage
Trie Cache	Fast lookup
Filter Layer	Remove unwanted suggestions

Key Design Decisions

Decision	Reason
Batch trie rebuild	Simpler and scalable
Cached top-k queries	Fast retrieval
Filter layer	Flexible moderation
Async deletion	Better performance

Summary

Trie operations are essential for maintaining an efficient autocomplete system.

The system supports:

- Trie creation using analytics data
- Periodic or direct trie updates
- Filtering and deletion of unsafe suggestions

Using:

- Workers
- Trie Cache
- Filter Layer
- Asynchronous processing

the autocomplete system achieves:

- Fast query responses
- Scalability

- Efficient updates
- Content moderation support.